

Domande Bio

Matias Bonoli

21 maggio 2026

Indice

- 1 Si spieghi come possiamo calcolare la distanza di edit tra due stringhe v e w utilizzando la griglia dove gli archi sono pesati. Come si ricostruisce una matrice di allineamento che riflette la distanza di edit? 2
- 2 Come si modifica il metodo di cui sopra per calcolare la distanza di Hamming (se v e w hanno stessa lunghezza) e la longest common subsequence tra v e w 6
- 3 In che cosa differisce l'allineamento globale dal locale e dal semi-globale anche in termini di calcolo dello stesso 11

Capitolo 1

Si spieghi come possiamo calcolare la distanza di edit tra due stringhe v e w utilizzando la griglia dove gli archi sono pesati. Come si ricostruisce una matrice di allineamento che riflette la distanza di edit?

La **distanza di edit** è definita come il numero minimo di operazioni elementari (delezione, inserzione, sostituzione) necessarie per trasformare una stringa nell'altra.

$$d_E(v, w) = \text{MIN numero di operazioni per trasformare } v \text{ in } w \quad (1.1)$$

Esempio di trasformazione

TGCATAT $\rightarrow$ ATCCGAT in 4 passi:

- TGCATAT $\rightarrow$ (inserisco **A** all'inizio)
- **A**TGCATAT $\rightarrow$ (sostituisco **C** al posto della 3° **G**)
- AT**C**CATAT $\rightarrow$ (sostituisco **G** al posto della 5° **A**)
- ATCC**G**TAT $\rightarrow$ (cancello la 6° **T**)
- ATCCG**A**T (fatto)

Per calcolare la distanza di edit $d_E(v, w)$ in modo efficiente si costruisce una **griglia di edit** (alignment grid) con archi pesati e si applica la **programmazione dinamica**.

Date due stringhe v e w , con $n = |v|$ e $m = |w|$ la **griglia di edit** è un grafo orientato $G = (V, E)$ di dimensioni $(n + 1) \times (m + 1)$, i cui vertici sono coppie (i, j) con $0 \leq i \leq n$ e $0 \leq j \leq m$. Ogni vertice, inoltre, rappresenta l'allineamento tra prefissi $v[1..i]$ e $w[1..j]$. Il vertice $(0, 0)$ è il **nodo sorgente** mentre (n, m) è lo **stato finale** (sink). Gli archi della griglia sono i movimenti permessi sulla griglia e sono etichettati mediante **funzione di costo**. Ogni cammino da $(0, 0)$ a (n, m) rappresenta un possibile allineamento fra v e w ,

mentre il costo complessivo rappresenta il costo dell'allineamento. Ogni cammino valido nella griglia da sorgente a sink corrisponde esattamente a un allineamento tra v e w . Il costo del cammino (somma dei pesi degli archi) è il numero di operazioni di editing.

I movimenti permessi e i loro pesi (costi) sono:

- **Spostamento verticale** da $(i-1, j)$ a (i, j) : corrisponde all'inserimento di un gap $(-)$ nella stringa w (delezione di v_i).
- **Spostamento orizzontale** da $(i, j-1)$ a (i, j) : corrisponde all'inserimento di un gap $(-)$ nella stringa v (inserimento di w_j).
- **Spostamento diagonale** da $(i-1, j-1)$ a (i, j) : corrisponde all'allineamento di v_i con w_j . (*match* se $v_i = w_j$, *sostituzione* se $v_i \neq w_j$).

Formalmente, la **funzione di costo** δ assegna un costo alla coppia di simboli che vengono allineati:

- **Match** (caratteri identici): $\delta(v_i, w_j) = 0$ se $v_i = w_j$ (ad esempio $\delta(A, A) = 0$), poiché non è richiesta alcuna operazione di modifica.
- **Mismatch** (sostituzione): $\delta(v_i, w_j) = 1$ se $v_i \neq w_j$ (ad esempio $\delta(A, C) = 1$).
- **Indel** (inserzione o delezione): $\delta(v_i, -) = 1$ per una delezione e $\delta(-, w_j) = 1$ per una inserzione.

Questi costi diventano i pesi degli archi nella griglia di edit:

- l'arco verticale da $(i-1, j)$ a (i, j) ha peso $\delta(v_i, -) = 1$;
- l'arco orizzontale da $(i, j-1)$ a (i, j) ha peso $\delta(-, w_j) = 1$;
- l'arco diagonale da $(i-1, j-1)$ a (i, j) ha peso $\delta(v_i, w_j)$, quindi 0 se $v_i = w_j$ e 1 se $v_i \neq w_j$.

Per calcolare il costo minimo per raggiungere un nodo (i, j) si applica la programmazione dinamica. Prima si definiscono i **casi base** (inizializzazione della prima riga e prima colonna):

- $S(i, 0) = i$ per $0 \leq i \leq n$
- $S(0, j) = j$ per $0 \leq j \leq m$

Successivamente, si riempie il resto della griglia con la seguente **equazione di ricorrenza**:

$$S(i, j) = \min \begin{cases} S(i-1, j) + 1 & \text{(Delezione)} \\ S(i, j-1) + 1 & \text{(Inserzione)} \\ S(i-1, j-1) + \delta(v_i, w_j) & \text{(Match/Mismatch)} \end{cases} \quad (1.2)$$

dove $\delta(v_i, w_j) = 0$ se $v_i = w_j$, e 1 altrimenti. Il valore calcolato nel nodo *sink*, ovvero $S(n, m)$, rappresenta la **distanza di edit ottima**. La **complessità** di calcolo, dovendo valutare un'operazione costante per ogni cella della griglia, è di $\mathcal{O}(n \times m)$ in tempo e in spazio.

Ricostruzione della matrice di allineamento

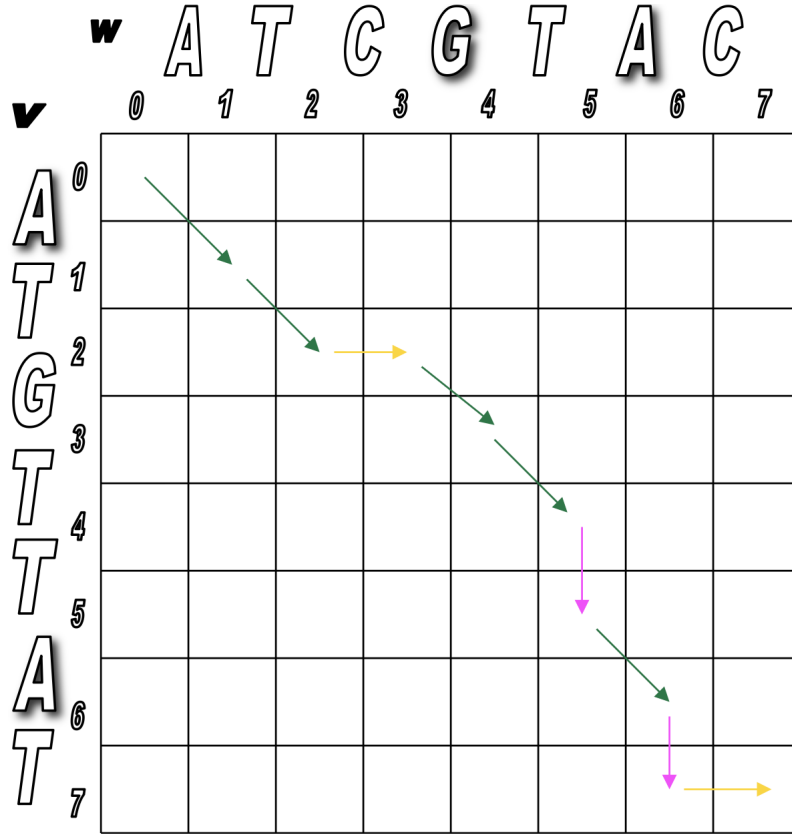


Figura 1.1: Didascalia dell'immagine

Per ricostruire una matrice di allineamento A (di dimensione $2 \times k$, con $k \geq \max(n, m)$) che rifletta tale distanza, si utilizza la procedura di **backtracking** (percorso a ritroso) all'interno della griglia:

1. Si parte dall'ultima cella, ovvero il nodo *sink* (n, m) .
2. Si risale la griglia all'indietro fino ad arrivare al nodo *sorgente* $(0, 0)$. Ad ogni passo, ci si sposta verso la cella adiacente che ha fornito il valore minimo usato nell'equazione di ricorrenza (in caso di parità tra più direzioni ottime, se ne può scegliere una qualsiasi, ottenendo allineamenti alternativi ma con lo stesso score ottimale).
3. Durante questa risalita, si costruisce la matrice di allineamento aggiungendo una colonna alla volta dall'ultima verso la prima:
 - Se lo spostamento a ritroso è **diagonale** verso $(i - 1, j - 1)$, si aggiunge la colonna $\begin{pmatrix} v_i \\ w_j \end{pmatrix}$.
 - Se lo spostamento a ritroso è **verticale** verso $(i - 1, j)$, il carattere v_i è allineato con un gap. Si aggiunge la colonna $\begin{pmatrix} v_i \\ - \end{pmatrix}$.
 - Se lo spostamento a ritroso è **orizzontale** verso $(i, j - 1)$, il carattere w_j è allineato con un gap. Si aggiunge la colonna $\begin{pmatrix} - \\ w_j \end{pmatrix}$.

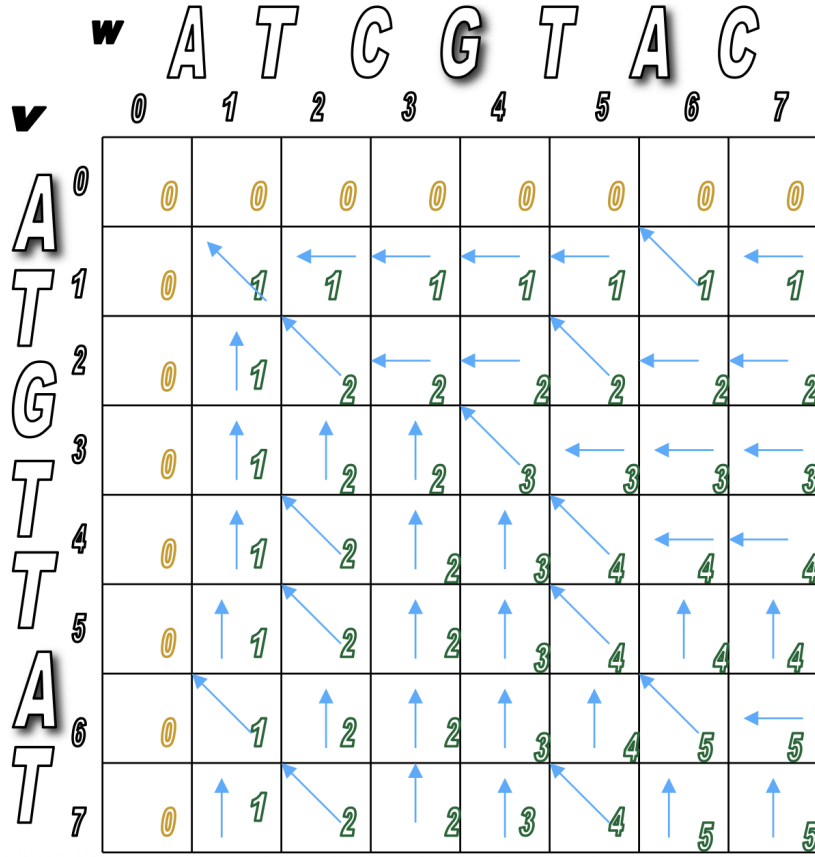


Figura 1.2: Didascalia dell'immagine

4. Il processo termina non appena si raggiunge la cella $(0, 0)$.

Le colonne così ottenute, lette nell'ordine inverso rispetto a come sono state ricostruite, formano la **matrice di allineamento globale**. Il **costo dell'allineamento** si calcola sommando il costo di ogni colonna della matrice:

$$\text{costo}(A) = \sum_{\ell=1}^k \delta(A_{1,\ell}, A_{2,\ell}) \quad (1.3)$$

dove ogni colonna $\begin{pmatrix} A_{1,\ell} \\ A_{2,\ell} \end{pmatrix}$ può rappresentare un match, un mismatch oppure un indel. Quindi una colonna con due caratteri uguali ha costo 0, mentre una colonna con due caratteri diversi oppure con un gap ha costo 1. La somma dei costi di tutte le colonne coincide con il costo del cammino nella griglia e quindi con la distanza di edit $d_E(v, w) = S(n, m)$.

Capitolo 2

Come si modifica il metodo di cui sopra per calcolare la distanza di Hamming (se v e w hanno stessa lunghezza) e la longest common subsequence tra v e w

Per adattare il metodo di calcolo della distanza di edit per ottenere la distanza di Hamming o Longest common subsequence (LCS), è necessario modificare le **operazioni consentite**, e di conseguenza la definizione della **funzione di costo** δ e l'algoritmo di **programmazione dinamica**.

Distanza di Hamming La **distanza di Hamming** si applica quando le due stringhe hanno la stessa lunghezza e si calcola confrontando i caratteri nelle stesse posizioni. Ogni posizione diversa contribuisce con costo 1, mentre ogni posizione uguale contribuisce con costo 0.

$$d_H(v, w) = \text{numero di posizioni in cui } v \text{ e } w \text{ differiscono} \quad (2.1)$$

Esempio

$$\begin{aligned} S_1 &= ACCGT \\ S_2 &= AGCGT \end{aligned}$$

Le due stringhe differiscono solo nella seconda posizione, dove $C \neq G$. Quindi:

$$d_H(S_1, S_2) = 1 \quad (2.2)$$

Se invece:

$$\begin{aligned} S_1 &= ACCGT \\ S_2 &= ACCGT \end{aligned}$$

non ci sono posizioni diverse, quindi:

$$d_H(S_1, S_2) = 0 \quad (2.3)$$

Per calcolare $d_H(v, w)$ utilizzando una griglia di edit definita per la distanza di edit, ma bisogna apportare alcune modifiche:

- La griglia di edit sarà una matrice di dimensione $n \times m$ con $n = m$ (poiché le stringhe devono avere la stessa lunghezza).
- Gli archi consentiti saranno solo quelli diagonali, poiché non sono permesse operazioni di inserzione o delezione.
- La funzione di costo δ sarà definita come:
 - **match**: $\delta(v_i, w_j) = 0$ se $v_i = w_j$
 - **mismatch**: $\delta(v_i, w_j) = 1$ se $v_i \neq w_j$
 - **indel**: $\delta(v_i, w_j) = n(\infty)$ se v_i o $w_j = -$

Il **costo** n assegnato agli indel serve a penalizzare fortemente inserzioni e delezioni. Infatti, poiché le due stringhe hanno la stessa lunghezza, il costo massimo della distanza di Hamming è n , cioè il caso in cui tutti i caratteri siano diversi. Se invece un allineamento usa un gap, per mantenere le due sequenze della stessa lunghezza dovrà introdurre almeno due indel, con costo totale almeno $2n$. Poiché $2n > n$, un allineamento con gap non potrà mai essere ottimo. In questo modo l'algoritmo è forzato ad allineare solo caratteri in **posizione corrispondente** (movimenti diagonali), e il costo finale coincide con il numero di mismatch.

Per calcolare il costo minimo si può usare ancora la programmazione dinamica, come nel caso della distanza di edit. Indichiamo con $S(i, j)$ il costo minimo per allineare il prefisso $v[1..i]$ con il prefisso $w[1..j]$.

I **casi base** diventano:

- $S(0, 0) = 0$
- $S(i, 0) = i \cdot n$ per $0 \leq i \leq n$
- $S(0, j) = j \cdot n$ per $0 \leq j \leq n$

Infatti, per allineare un prefisso non vuoto con una stringa vuota, è necessario inserire solo gap, e ogni gap ha costo n .

Successivamente, si riempie il resto della griglia con la seguente **equazione di ricorrenza**:

$$S(i, j) = \min \begin{cases} S(i-1, j) + n & \text{(Delezione)} \\ S(i, j-1) + n & \text{(Inserzione)} \\ S(i-1, j-1) + \delta(v_i, w_j) & \text{(Match/Mismatch)} \end{cases} \quad (2.4)$$

dove:

$$\delta(v_i, w_j) = \begin{cases} 0 & \text{se } v_i = w_j \\ 1 & \text{se } v_i \neq w_j \end{cases} \quad (2.5)$$

Il valore nel nodo finale:

$$S(n, n) \quad (2.6)$$

coincide quindi con la distanza di Hamming:

$$S(n, n) = d_H(v, w) \quad (2.7)$$

La **matrice di allineamento** che si ottiene non contiene gap, ma solo colonne del tipo:

$$\begin{pmatrix} v_i \\ w_i \end{pmatrix}$$

per ogni posizione i . Ogni colonna ha costo 0 se i due simboli sono uguali e costo 1 se sono diversi. La somma dei costi delle colonne è quindi esattamente il numero di mismatch tra v e w . In questo modo, la distanza di Hamming può essere vista come un caso particolare della distanza di edit in cui inserzioni e delezioni sono rese non convenienti tramite un costo molto alto.

Longest Common Subsequence (LCS) Date due sequenze $v = v_1v_2 \dots v_n$ e $w = w_1w_2 \dots w_m$ la **longest common subsequence (LCS)** di v e w è la sequenza di posizioni $1 \leq i_1 < i_2 < \dots < i_k \leq n$ e $1 \leq j_1 < j_2 < \dots < j_k \leq m$ tale che la i -esima posizione di v e la j -esima posizione di w contengono lo stesso simbolo, ovvero $v_{i_t} = w_{j_t}$ per ogni $t = 1, 2, \dots, k$. La lunghezza della longest common subsequence è indicata con $LCS(v, w)$.

Esempio

Consideriamo:

$$\begin{aligned} v &= ATCTGATC \\ w &= TGCATAC \end{aligned}$$

Un possibile allineamento tra v e w è:

$$A = \begin{pmatrix} A & T & - & C & - & T & G & A & T & C \\ - & T & G & C & A & T & - & A & - & C \end{pmatrix}$$

Le colonne evidenziate in rosso sono i **match**, cioè le colonne in cui i due simboli allineati sono uguali. Il cammino corrispondente nella griglia è:

$$\begin{aligned} (0, 0) &\rightarrow (1, 0) \rightarrow (2, 1) \rightarrow (2, 2) \rightarrow (3, 3) \rightarrow (3, 4) \\ &\rightarrow (4, 5) \rightarrow (5, 5) \rightarrow (6, 6) \rightarrow (7, 6) \rightarrow (8, 7) \end{aligned}$$

Quindi le posizioni scelte in v sono:

$$2 < 3 < 4 < 6 < 8$$

e le posizioni corrispondenti in w sono:

$$1 < 3 < 5 < 6 < 7$$

La **sottosequenza comune** letta nelle colonne di match è:

$$TCTAC$$

In questo esempio la lunghezza della longest common subsequence è quindi:

$$LCS(v, w) = 5$$

Anche per LCS è possibile utilizzare una **griglia di edit** di dimensione $(n+1) \times (m+1)$, con $n = |v|$ e $m = |w|$, con archi pesati e applicare la **programmazione dinamica**. Tuttavia, come per la distanza di Hamming, è necessario modificare operazioni consentite, funzione di costo δ e algoritmo di programmazione dinamica. In particolare dovendo trovare le posizioni dove v e w hanno simboli uguali, andrà considerato il **movimento diagonale**, solo in caso di match. Inoltre al contrario di edit e hamming serve trovare la più lunga sottosequenza comune, quindi servirà modificare i valori di costo per riflettere questa differenza. andando ad escludere i mismatch dall'allineamento.

Quindi la **funzione di costo** δ sarà definita come:

- **match:** $\delta(v_i, w_j) = 1$ se $v_i = w_j$ (l'arco diagonale da $(i-1, j-1)$ a (i, j))
- **Indel** (inserzione o delezione): $\delta(v_i, -) = 0$ per una delezione (l'arco verticale da $(i-1, j)$ a (i, j)) e $\delta(-, w_j) = 0$ per una inserzione (l'arco orizzontale da $(i, j-1)$ a (i, j)).

Per calcolare la LCS fra due sequenze si utilizza la programmazione dinamica ma cambia l'obiettivo: invece di minimizzare il costo, si massimizza il numero di match. non cerca più di minimizzare il costo totale, ma di massimizzare il numero di match (lunghezza della sottosequenza comune).+

Indichiamo con $S(i, j)$ la lunghezza della LCS tra il prefisso $v[1..i]$ e il prefisso $w[1..j]$.

I **casi base** diventano:

- $S(0, 0) = 0$
- $S(i, 0) = 0$ per $0 \leq i \leq n$
- $S(0, j) = 0$ per $0 \leq j \leq m$

Infatti, se una delle due sequenze è vuota, non esiste alcuna sottosequenza comune non vuota, quindi la lunghezza della LCS è 0.

Si riempie il resto della griglia con la seguente **equazione di ricorrenza**:

$$S(i, j) = \max \begin{cases} S(i-1, j) & (\text{salto } v_i) \\ S(i, j-1) & (\text{salto } w_j) \\ S(i-1, j-1) + 1 & \text{se } v_i = w_j \end{cases} \quad (2.8)$$

Il valore nel nodo finale:

$$S(n, n) \quad (2.9)$$

coincide quindi con la lunghezza della longest common subsequence:

$$S(n, m) = LCS(v, w) \quad (2.10)$$

Effettuando il **backtracking** nella griglia, partendo dal nodo finale (n, m) e risalendo fino al nodo sorgente $(0, 0)$, è possibile ricostruire una matrice di allineamento A .

Per ottenere la **longest common subsequence**, si considerano solo le colonne di A che corrispondono a un match, cioè colonne del tipo:

$$\begin{pmatrix} v_i \\ w_j \end{pmatrix}$$

con $v_i = w_j$.

I simboli letti in queste colonne, nell'ordine dell'allineamento, formano la longest common subsequence.

Capitolo 3

In che cosa differisce l'allineamento globale dal locale e dal semi-globale anche in termini di calcolo dello stesso

Siano $V = v_1v_2 \dots v_n$ e $W = w_1w_2 \dots w_m$ due stringhe di lunghezza n e m definite su un alfabeto Σ . Si indica con $-$ il simbolo di gap e con $\delta : (\Sigma \cup \{-\}) \times (\Sigma \cup \{-\}) \rightarrow \mathbb{N}$ una funzione di costo che assegna un costo a ogni coppia di simboli (compresi i gap), in particolare:

- $\delta(v_i, w_j) = k$ se $v_i = w_j$ (match)
- $\delta(v_i, w_j) = s$ se $v_i \neq w_j$ (mismatch)
- $\delta(v_i, -) = \delta(-, w_j) = d$

Un allineamento tra V e W è una matrice A di dimensione $2 \times k$ (con $k \geq \max(n, m)$) ottenuta inserendo eventualmente i gap nelle sequenze, in modo tale che, eliminando i gap dalla prima riga, si ottiene V , ed eliminando i gap dalla seconda riga, si ottiene W . Ogni colonna di A rappresenta un match, mismatch, inserzione o delezione. Il punteggio di allineamento è dato dalla somma dei punteggi delle singole colonne

$$C(A) = \sum_{i=1}^k \delta(A_{1,i}, A_{2,i}) \quad (3.1)$$

Per calcolare un allineamento ottimo si usa una griglia di edit di dimensione $(n+1) \times (m+1)$, in cui $C(i, j)$ rappresenta lo score ottimo per allineare il prefisso $v[1..i]$ con il prefisso $w[1..j]$.

Allineamento Globale L'allineamento globale di due sequenze V e W è un allineamento che considera entrambe le sequenze nella loro interezza. Quindi l'obiettivo è trovare la matrice di allineamento A di score ottimo tra tutta la sequenza V e tutta la sequenza W .

Nel calcolo tramite programmazione dinamica, $S(i, j)$ rappresenta lo score ottimo di allineamento globale fra il prefisso V_i e il prefisso W_j .

I casi base sono:

- $S(0, 0) = 0$
- $S(i, 0) = i \cdot d$
- $S(0, j) = j \cdot d$

Dove d è il costo dell'inserimento di un gap. Infatti, per allineare un prefisso non vuoto con una stringa vuota, è necessario inserire solo gap, e ogni gap ha costo d .

per $i > 0$ e $j > 0$ la ricorrenza è:

$$S(i, j) = \min(\max) \begin{cases} S(i-1, j-1) + \delta(v_i, w_j) & \text{(match/mismatch)} \\ S(i-1, j) + \delta(v_i, -) & \text{(delezione)} \\ S(i, j-1) + \delta(-, w_j) & \text{(inserzione)} \end{cases} \quad (3.2)$$

Poichè si considerano entrambe le sequenze nella loro interezza, il punteggio ottimo si trova nel nodo finale $S(n, m)$.

La ricostruzione dell'allineamento avviene tramite backtracking, partendo da $S(n, m)$ e risalendo fino a $S(0, 0)$, seguendo i movimenti che hanno portato al punteggio ottimo. Si ottiene così la matrice di allineamento A .

Allineamento Locale Date due stringe:

$$V = v_1 v_2 \dots v_{i-1} v_i v_{i+1} \dots v_n \quad \text{e} \quad W = w_1 w_2 \dots w_{j-1} w_j w_{j+1} \dots w_m$$

un **allineamento locale** consiste nel determinare il miglior allineamento globale tra una sottostringa di V e una sottostringa di W .

Quindi, a differenza dell'allineamento globale, non si vuole necessariamente allineare tutta la sequenza V con tutta la sequenza W , ma si vogliono individuare le regioni più simili contenute nelle sequenze.

Formalmente, l'obiettivo è quindi di trovare due sotto stringe $V' = v_i v_{i+1} \dots v_{i+k}$ e $W' = w_j w_{j+1} \dots w_{j+k}$ (con $k \geq 0$) tali che lo score di allineamento globale fra le due sottostringhe sia massimi fra tutte le possibile coppie di sottostringhe in V e W .

Anche per l'allineamento locale si utilizza la griglia di edit S di dimensione: $(n+1) \times (m+1)$, dove però $S(i, j)$ rappresenterà il miglior score di allineamento globale tra due sottostringhe che terminano rispettivamente in v_i e w_j .

In altre parole, $S(i, j)$ non rappresenta lo score tra prefissi di V e W , ma il miglior score tra tutte le coppie di sottostringhe che terminano in v_i e w_j . Lo score dell'allineamento locale tra V e W sarà quindi il massimo valore presente nella matrice S .

I **casi base** sono:

- $S(0, 0) = 0$
- $S(i, 0) = 0$ per $0 \leq i \leq n$
- $S(0, j) = 0$ per $0 \leq j \leq m$

A differenza dell'allineamento globale, qui non si inizializza la prima riga e la prima colonna con il costo dei gap, in quanto le parti iniziali delle sequenze che non appartengono alla sottostringa ottima possono essere ignorate. L'allineamento, infatti, può iniziare in qualunque punto della matrice.

Per $i > 0$ e $j > 0$ la ricorrenza è:

$$S(i, j) = \max(\min) \begin{cases} 0 \\ S(i-1, j-1) + \delta(v_i, w_j) \\ S(i-1, j) + \delta(v_i, -) \\ S(i, j-1) + \delta(-, w_j) \end{cases} \quad (3.3)$$

Il primo caso, con punteggio 0, è la differenza principale rispetto all'allineamento globale. Infatti, se tutte le opzioni di allineamento hanno un punteggio negativo, è preferibile non allineare nulla quindi interrompere l'allineamento corrente e iniziare un nuovo allineamento a partire dalla cella successiva.

Una volta riempita la matrice S , lo score ottimo dell'allineamento locale non si trova necessariamente in $S(n, m)$ ma è:

$$\max_{0 \leq i \leq n, 0 \leq j \leq m} S(i, j)$$

cioè il massimo valore presente nella matrice.

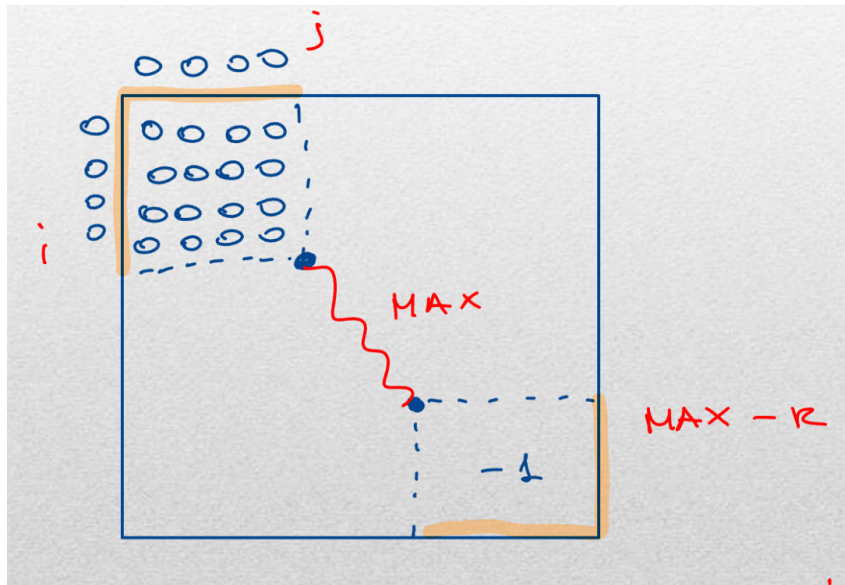


Figura 3.1: Didascalia dell'immagine

La ricostruzione dell'allineamento avviene tramite backtracking, partendo dalla cella che contiene il punteggio massimo. Il backtracking procede a ritroso seguendo i movimenti che hanno portato al punteggio ottimo e si ferma quando raggiunge una cella con punteggio 0, che rappresenta l'inizio dell'allineamento locale ottimo.

Allineamento Semi-globale L'allineamento semi-globale di due sequenze $V = v_1 v_2 \dots v_m$ e $W = w_1 w_2 \dots w_n$ è una famiglia di problemi di allineamento in cui non si

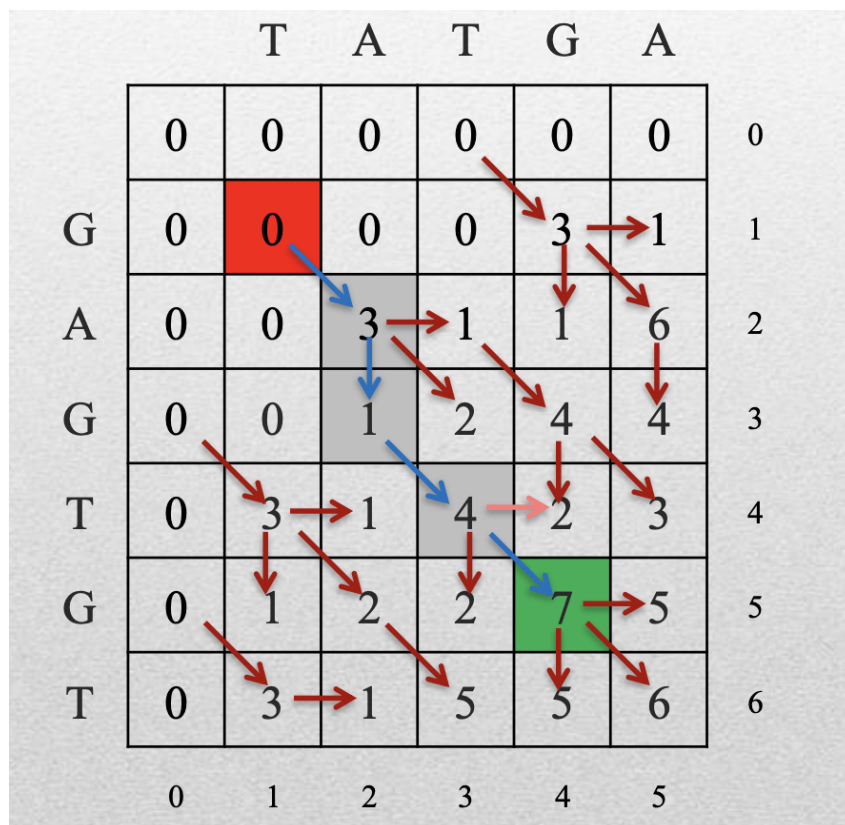


Figura 3.2: Didascalia dell'immagine

richiede necessariamente di allineare entrambe le sequenze per intero, come nell'allineamento globale, ma non si cercano nemmeno due sottostringhe completamente arbitrarie, come nell'allineamento locale.

L'idea generale è cercare un allineamento globale ottimo tra due porzioni vincolate delle sequenze. Questo significa che il problema specifica che tipo di porzione si può scegliere: un prefisso, un suffisso, una sottostringa, oppure l'intera sequenza. Una volta scelta tale porzione, essa viene allineata globalmente con l'altra porzione.

Quindi, in generale, l'allineamento semi-globale cerca un allineamento globale ottimo tra $V[a..b]$ e $W[c..d]$, dove però gli indici a, b, c, d non sono completamente liberi come nel locale, ma sono vincolati dal tipo di problema, in particolare:

- se si cerca un prefisso di V , allora la porzione deve iniziare da v_1 , quindi è del tipo $V[1..b]$;
- se si cerca un suffisso di W , allora la porzione deve terminare in w_n , quindi è del tipo $W[c..n]$;
- se si cerca una sottostringa di V , allora la porzione può iniziare e finire in posizioni interne, quindi è del tipo $V[a..b]$;
- se si considera tutta W , allora la porzione è fissata ed è $W[1..n]$.

La stessa cosa vale per la sequenza W .

Dal punto di vista del calcolo, anche l'allineamento semi-globale usa una matrice dinamica S di dimensione:

$$(m + 1) \times (n + 1)$$

La ricorrenza interna è in genere la stessa dell'allineamento globale:

$$S(i, j) = \max \begin{cases} S(i - 1, j - 1) + \delta(v_i, w_j) \\ S(i - 1, j) + \delta(v_i, -) \\ S(i, j - 1) + \delta(-, w_j) \end{cases}$$

La differenza rispetto al globale sta nel fatto che non sempre l'ottimo viene cercato nella cella finale $S(m, n)$. A seconda del tipo di porzione considerata, lo score ottimo può essere cercato nell'ultima riga, nell'ultima colonna, in tutta la matrice o in un'altra regione specifica.

Anche l'**inizializzazione** e il **punto di arresto del backtracking** dipendono dal problema semi-globale considerato: le parti della sequenza che possono essere ignorate non vengono penalizzate, mentre le parti che devono essere effettivamente allineate vengono trattate come in un allineamento globale.

1. Prefisso di V contro tutta W

$$V[1..r] \quad \text{contro} \quad W[1..m]$$

Lo score ottimo si cerca nell'ultima colonna:

$$\max_{0 \leq i \leq n} S(i, m)$$

perché W deve essere considerata interamente, mentre il prefisso di V può terminare in una qualunque posizione i .

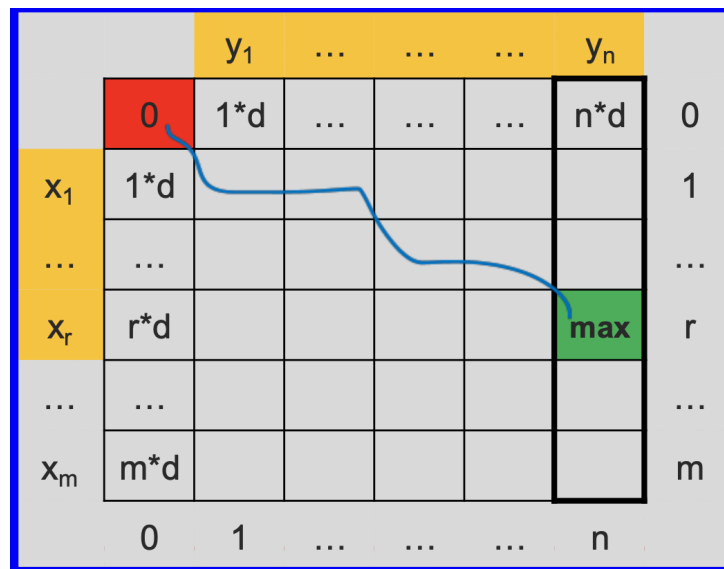


Figura 3.3: Allineamento tra un prefisso di V e tutta W .

2. Tutta V contro prefisso di W

$$V[1..n] \quad \text{contro} \quad W[1..c]$$

Lo score ottimo si cerca nell'ultima riga:

$$\max_{0 \leq j \leq m} S(n, j)$$

perché V deve essere considerata interamente, mentre il prefisso di W può terminare in una qualunque posizione j .

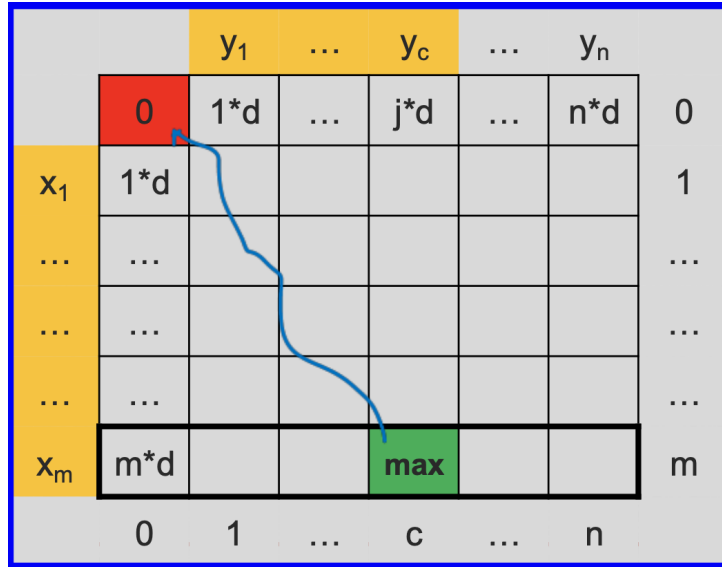


Figura 3.4: Allineamento tra tutta V e un prefisso di W .

3. Prefisso di V contro prefisso di W

$$V[1..r] \quad \text{contro} \quad W[1..c]$$

Lo score ottimo si cerca in tutta la matrice:

$$\max_{0 \leq i \leq n, 0 \leq j \leq m} S(i, j)$$

perché entrambi i prefissi possono terminare in posizioni non fissate.

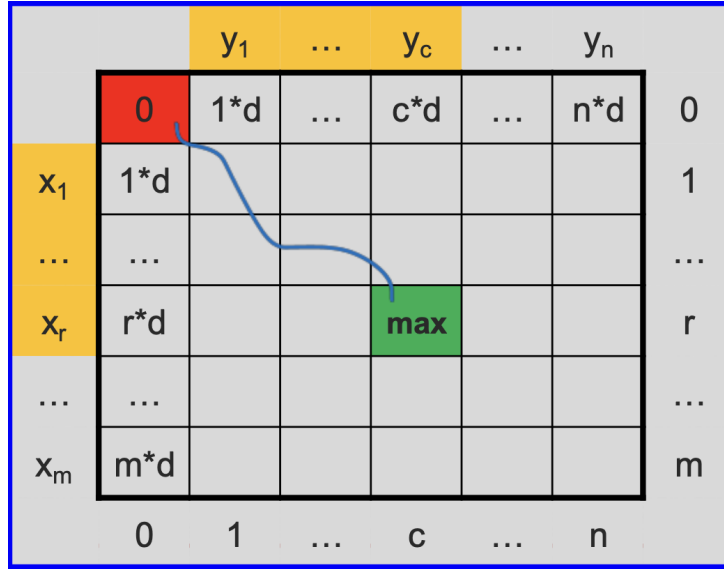


Figura 3.5: Allineamento tra un prefisso di V e un prefisso di W .

4. Suffisso di V contro suffisso di W

$$V[a..n] \quad \text{contro} \quad W[c..m]$$

Lo score ottimo si trova in:

$$S(n, m)$$

perché entrambi i suffissi devono terminare alla fine delle rispettive sequenze. Le parti iniziali non appartenenti ai suffissi possono invece essere ignorate.

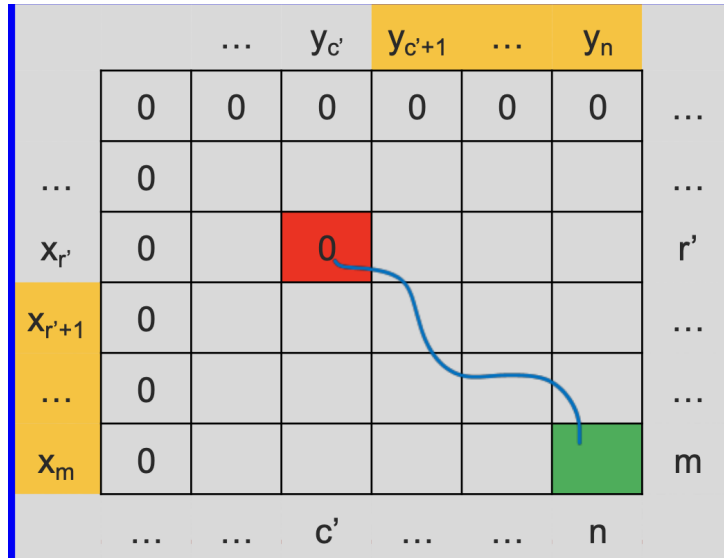


Figura 3.6: Allineamento tra un suffisso di V e un suffisso di W .

5. Suffisso di V contro prefisso di W

$$V[a..n] \quad \text{contro} \quad W[1..c]$$

Lo score ottimo si cerca nell'ultima riga:

$$\max_{0 \leq j \leq m} S(n, j)$$

perché il suffisso di V deve terminare in v_n , mentre il prefisso di W può terminare in una qualunque posizione j .

		y_1	...	y_c	...	y_n		
		0	$1*d$	$2*d$	...	...	$n*d$	...
x_1		0						...
...		0						...
...		...						...
...		...						...
x_m		0			max			m
		...	...	c	...	n		

Figura 3.7: Allineamento tra un suffisso di V e un prefisso di W .

6. Sottstringa di V contro tutta W

$$V[a..b] \quad \text{contro} \quad W[1..m]$$

Lo score ottimo si cerca nell'ultima colonna:

$$\max_{0 \leq i \leq n} S(i, m)$$

perché W deve essere usata tutta, mentre la sottstringa di V può terminare in una qualunque posizione i .

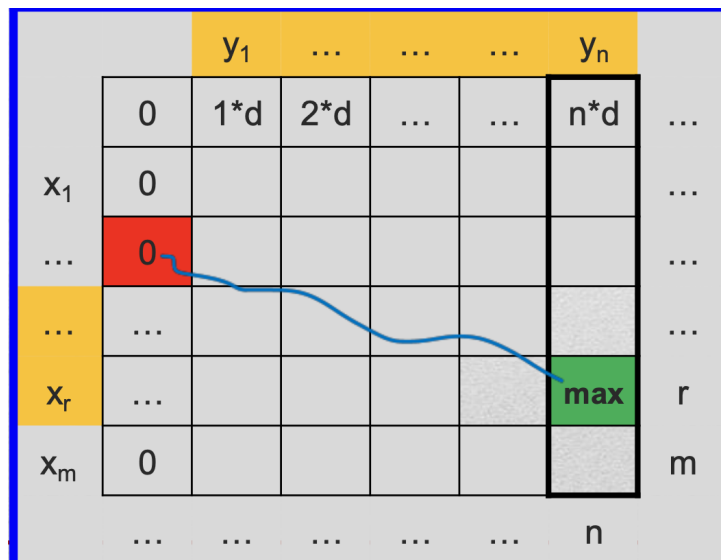


Figura 3.8: Allineamento tra una sottostringa di V e tutta W .